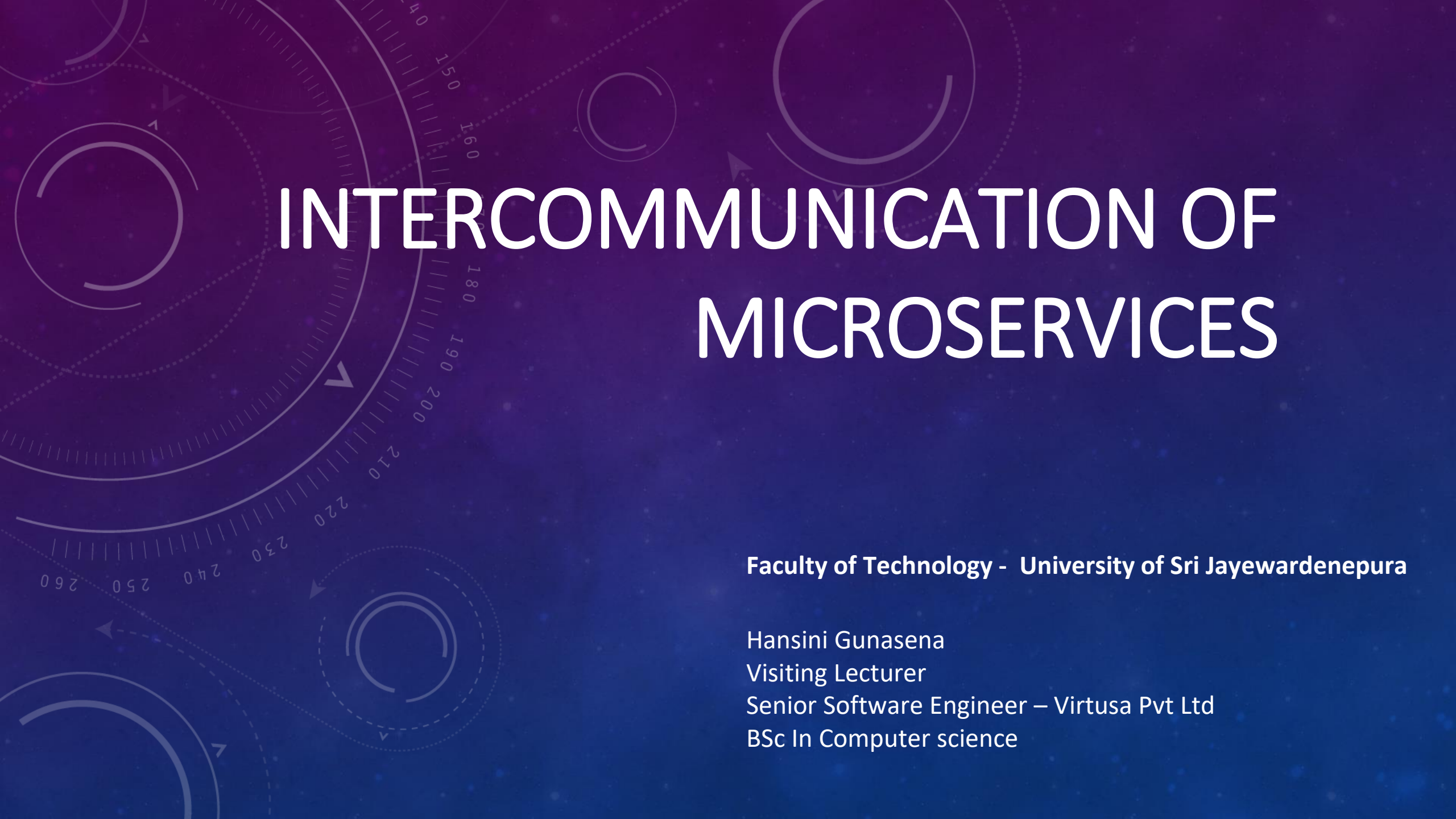




INTERCOMMUNICATION OF MICROSERVICES

Faculty of Technology - University of Sri Jayewardenepura

Hansini Gunasena

Visiting Lecturer

Senior Software Engineer – Virtusa Pvt Ltd

BSc In Computer science



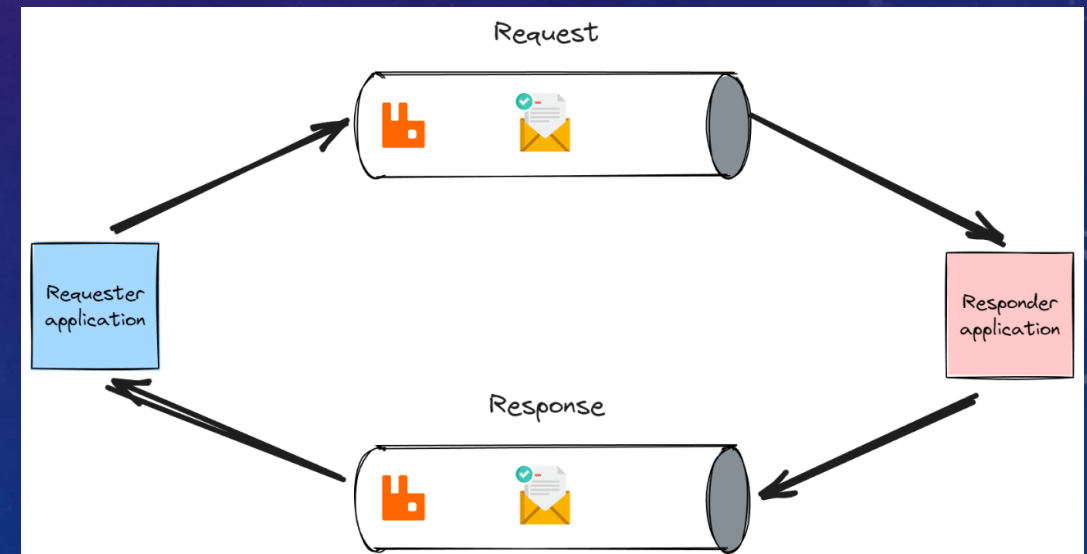
COMMUNICATION PATTERNS IN MICROSERVICES

- Communication patterns in microservices define how services interact with each other to achieve specific tasks. These patterns help manage the complexity of distributed systems, ensuring that services communicate efficiently and reliably. Below are some common communication patterns in microservices:

- **1. Request-Response Pattern**
- **2. Publish-Subscribe Pattern:**
- **3. Event Sourcing Pattern**
- **4. CQRS (Command Query Responsibility Segregation) Pattern**
- **5. Circuit Breaker Pattern**
- **6. Saga Pattern**
- **7. API Gateway Pattern**
- **8. Backend for Frontend (BFF) Pattern**
- **9. Strangler Pattern**
- **10. Service Mesh Pattern**

1. REQUEST-RESPONSE PATTERN

- One service sends a request and waits for a response (synchronous).
 - **Use Case:** Immediate replies like fetching user details or processing payments.
 - **Examples:** REST, gRPC, SOAP.
-
- The **Request-Response Pattern** is the most common communication style in microservices where:
 - **Service A sends a request to**
 - **Service B, and**
 - **Service B sends back a response** before Service A continues.
 - It is a **synchronous** communication method.



Client → (Request) → Service
Service → (Response) → Client

Order Service → (REST call) → Inventory Service
Inventory Service → (stock details) → Order Service

Key Characteristics

1 Synchronous

- The caller **waits** for the response.
- Execution is **blocked** until the callee replies.

2 Real-time Interaction

Useful when you need immediate results:

- check stock
- verify payment
- fetch user details

3 Point-to-point

Only two services are involved in the transaction.

Advantages

- ✓ Simple and easy to understand
- ✓ Easy to implement using REST or gRPC
- ✓ Good for real-time decisions
- ✓ Works well for small to medium traffic

Disadvantages

- ✗ Caller is blocked → slow downstream = slow upstream
- ✗ Tight coupling between services
- ✗ If Service B is down, Service A fails
- ✗ Harder to scale for high-traffic workloads
- ✗ Increases overall latency

Technologies Used for Request–Response

- **REST API (HTTP + JSON)** – most common
- **gRPC** – faster, binary protocol
- **GraphQL** – for flexible data retrieval
- **Feign Client / WebClient** – used in Spring Boot

When to Use Request–Response Pattern?

Use when:

- ✓ You need instant results
- ✓ The operation must return data immediately
- ✓ The services are reliable and low latency

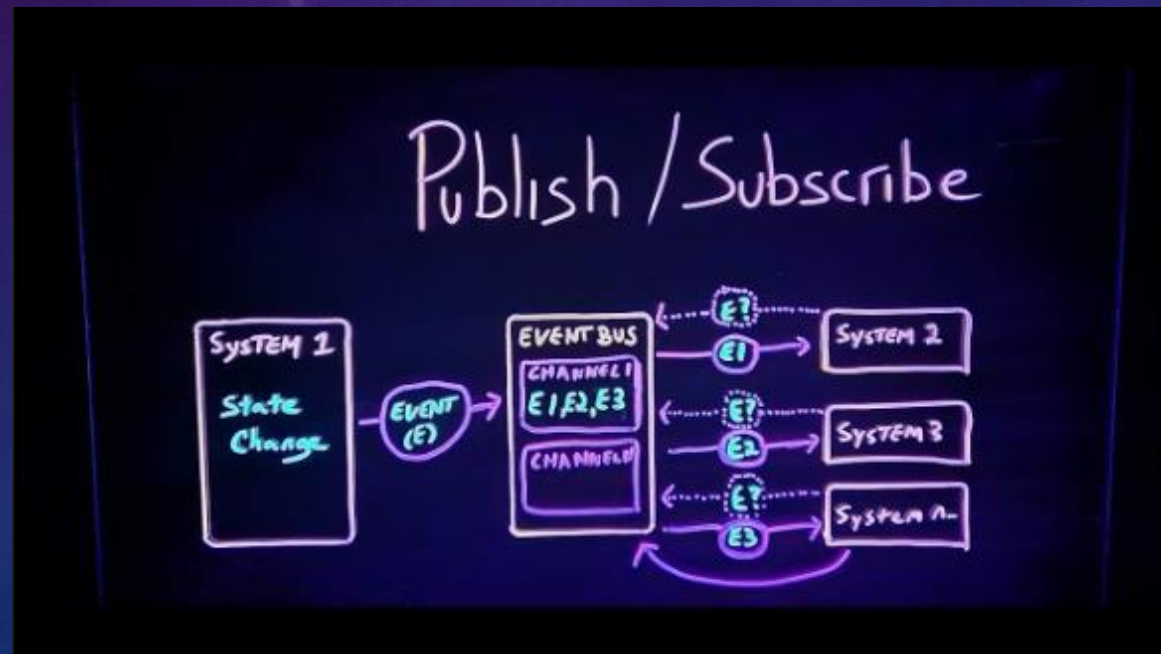
Avoid when:

- ✗ Very high traffic
- ✗ Long-running operations
- ✗ When failure of one service should not block others

“Request–Response is a synchronous communication pattern where a service waits for the result of another service before proceeding.”

2. PUBLISH-SUBSCRIBE PATTERN

- Services publish events to a broker; subscribers react asynchronously.
- **Use Case:** Notify multiple services of events (e.g., new user registration).
- **Examples:** Kafka, AWS SNS, Google Pub/Sub.



The **Publish/Subscribe pattern** (Pub/Sub) is an **asynchronous communication** method where:

- A service (publisher) **publishes an event/message**
- The message goes to a **topic**
- Multiple other services (subscribers) **receive and react** to that event

Publishers and subscribers **do NOT know each other** → loose coupling.



Order Service publishes: order_created event

- Payment Service consumes it
- Notification Service consumes it
- Analytics Service consumes it

One event → many reactions.

Key Characteristics

1 Asynchronous

- Publishers don't wait for subscribers.
No blocking, high performance.

2 Loose Coupling

- Publishers don't need to know:
- who is listening
- how many subscribers exist
- whether subscribers are running

3 Scalable

- New subscribers can be added without changing the publisher.

4 Event-driven

- Actions occur **because something happened** (an event).

Common Tools

- Apache Kafka (most popular)
- RabbitMQ
- Google Pub/Sub
- AWS SNS/SQS
- Redis Streams

Why Use Pub/Sub?

- Avoid tight coupling between microservices
- Handle high event volume (e.g., orders, payments)
- Make systems more scalable and fault-tolerant
- Build event-driven architectures
- Allow multiple services to react to the same event

Typical Use Cases

- Sending notifications
- Order creation → trigger multiple workflows
- Logging, analytics, auditing
- Payment events
- IoT sensor data
- Real-time streaming systems

✓ Advantages

- High scalability
- Super fast and non-blocking
- Subscribers can be added anytime
- System remains stable even if some services are down
- Great for distributing events to many services

✗ Disadvantages

- More complex to implement
- Harder to debug (since async)
- No immediate response → **eventual consistency**
- Requires message brokers

The Publish/Subscribe pattern is an asynchronous, event-driven communication model where one service publishes events and multiple services subscribe to and react to them.

API GATEWAY PATTERN

- The **API Gateway Pattern** is a design pattern where all clients (web apps, mobile apps, external systems) connect to the system through **one single entry point** instead of calling microservices directly.
- It sits in front of all microservices like a “front door.”

🌀 1. Why Do We Need an API Gateway?

Without a gateway:

- Clients must call many microservices individually
- They must know all microservice URLs
- Versioning is difficult
- Authentication needs to be repeated for every service
- Network calls increase (performance issues)

With a gateway:

- Clients call **ONE URL only**
- Gateway routes requests to the right microservice
- Authentication, logging, monitoring, rate limiting → done in one place
- Fewer round trips
- Provides separation and security

2. What an API Gateway Does

✓ Request Routing

Client → /products → Gateway → Product Service

✓ Aggregation (Fan-in)

Gateway can combine responses from multiple microservices

Example:

Client wants **Product + Reviews + Inventory**

Gateway collects them → sends one combined response.

✓ Cross-Cutting Concerns

Gateway handles common tasks:

- Authentication & Authorization
- Logging
- Monitoring & tracing
- Rate limiting
- Request/response transformation
- Caching

✓ Security

Hides internal microservice endpoints from external clients.

- <https://www.youtube.com/watch?v=6ULyXuHKxg8&t=21s>

3. Where It Fits in Microservices

mathematica

Client (Web/Mobile)

|
v



← Single Entry Point

/ | \
User Product Order ... (many services)
Service Service Service

4. Example Students Can Understand

Imagine a food delivery app

Without Gateway:

Mobile app must call:

- Restaurant service
- Delivery service
- User service
- Payment service

→ Complex, slow, inconsistent.

With Gateway:

Mobile app GET /order-details

The gateway fetches:

calls **ONE endpoint**, example:

Order service

- Restaurant service
- Rider service
- Payment service

And returns a single combined response.

5. Advantages of API Gateway

✓ Centralized access point

Clients use only ONE base URL.

✓ Better security

Internal services stay private.

✓ Simplified client logic

Clients don't need to know microservices structure.

✓ Reduced round trips

Supports response aggregation.

✓ Helps with versioning

Gateway routes `/v1/*` and `/v2/*` separately.

6. Disadvantages

✗ A single point of failure

Needs proper load balancing & scaling.

✗ Adds extra latency

One more hop, but usually small.

✗ More complexity

Needs configuration, monitoring, and DevOps support.

7. Popular API Gateway Tools

For production:

- Kong API Gateway
- Apigee (Google)
- AWS API Gateway
- Azure API Management
- Traefik
- NGINX API Gateway

For Spring Boot projects:

- Spring Cloud Gateway